INTELLIGENT SYSTEM DEVELOPMENT

Knowledge Graph and Retrieval-Augmented Generation

A Hands-on Tutorial for Building
Domain-Specific Intelligent Chat Systems

From Deep Learning Models to Knowledge-Based AI

Lecture and Laboratory Tutorial

2026

Contents

1 Lecture Overview **3**
1.1 Learning Outcomes . . . 3
1.2 Recommended Environment . . . 4

2 From Deep Learning to Intelligent Systems **5**
2.1 Prediction Versus Knowledge . . . 5
2.2 The Intelligent-System Perspective . . . 5

3 Knowledge Representation **7**
3.1 Structured Knowledge . . . 7
3.2 Unstructured Knowledge . . . 7
3.3 Why We Need Both . . . 8

4 Knowledge Graph Fundamentals **9**
4.1 Knowledge Graph as Triples . . . 9
4.2 Python Implementation . . . 9

5 Building a Course Knowledge Graph **11**
5.1 Graph Query . . . 12

6 Embeddings and Semantic Representation **14**
6.1 Cosine Similarity . . . 14
6.2 First Experiment: TF-IDF . . . 14

7 Semantic Embeddings with Sentence Transformers **16**

8 Document Chunking **18**
8.1 Simple Chunking Algorithm . . . 18
8.2 Why Overlap? . . . 19

9 Document Metadata **20**
9.1 A Better Internal Representation . . . 20

10 What is Retrieval-Augmented Generation? **22**
10.1 Why RAG? . . . 22

11 Building the First RAG Retriever **24**
11.1 Retrieval Function . . . 25

12 Constructing the RAG Context **27**

13 Connecting an LLM **29**

14 Complete Command-Line RAG Prototype **31**

15 Knowledge Graph + RAG **35**
15.1 Graph Retrieval . . . 35
15.2 Combining the Two Sources . . . 36

16 Concrete Application: Personal Course Assistant **37**
16.1 Suggested Project Directory 37

17 Building a Chat Interface **39**

18 Hallucination and Grounding **41**
18.1 Three Important Failure Modes 41
18.1.1 Retrieval Failure 41
18.1.2 Generation Failure 41
18.1.3 Knowledge Gap 41
18.2 Grounding Instruction 42

19 Evaluation of the RAG System **43**
19.1 Test Set 43
19.2 Retrieval Recall 43
19.3 Answer Evaluation 43

20 Final Student Project **45**
20.1 Minimum Architecture 45
20.2 Advanced Architecture 45
20.3 Required Deliverables 46

21 Recommended Student Report **47**
21.1 1. Introduction 47
21.2 2. Domain and Knowledge Sources 47
21.3 3. Knowledge Graph 47
21.4 4. RAG Architecture 47
21.5 5. Implementation 48
21.6 6. Experiments 48
21.7 7. Evaluation 48
21.8 8. Deployment 48
21.9 9. Limitations 48
21.1010. Conclusion 48

22 Laboratory Exercises **49**

23 Debugging and System Design Checklist **51**
23.1 Problem 1: Poor Retrieval 51
23.2 Problem 2: Good Retrieval, Bad Answer 51
23.3 Problem 3: Hallucination 51
23.4 Problem 4: Graph Retrieval Too Broad 51
23.5 Problem 5: Slow Retrieval 52
23.6 Problem 6: Security 52

24 Final System Architecture **53**

25 From ML and DL to RAG-Based Intelligent Systems **55**
25.1 The Main Concept 55
25.2 Student Final Checklist 55

1 Lecture Overview

This lecture is designed as the next stage after students have studied machine learning and deep learning applications such as:

- diabetes prediction;
- MNIST image classification;
- house-price prediction;
- stock-price prediction;
- gold-price prediction;
- CNN-based image classification;
- RNN and LSTM sequence modelling.

The central transition in this lecture is:

> **From learning patterns from data to developing intelligent systems that can use knowledge.**

The objective is not simply to teach students how to call an LLM. The objective is to understand the architecture of a modern knowledge-based intelligent system and then implement a small domain-specific conversational application.

1.1 Learning Outcomes

After completing this lecture, students should be able to:

1. explain why prediction models alone are not complete intelligent systems;
2. distinguish structured and unstructured knowledge;
3. construct a simple Knowledge Graph;
4. represent knowledge using entities and relations;
5. understand document chunking;
6. understand embeddings;
7. implement semantic retrieval;
8. explain Retrieval-Augmented Generation (RAG);
9. implement a basic RAG system;
10. combine Knowledge Graph retrieval with vector retrieval;
11. evaluate retrieval quality;
12. evaluate answer quality;
13. build a simple conversational interface;
14. deploy a small domain-specific intelligent chatbot.

1.2 Recommended Environment

The practical examples use Python and are designed to be executable on a normal CPU computer.

Create an environment:

```
python -m venv rag_env
```

On Windows:

```
rag_env\Scripts\activate
```

Install the basic packages:

```
pip install numpy pandas scikit-learn networkx
pip install sentence-transformers streamlit
```

The tutorial deliberately separates the retrieval system from the LLM provider. Therefore, students can later connect:

- an approved cloud LLM API;
- a local language model;
- an institutional model server;
- another compatible generation service.

The retrieval architecture remains unchanged.

2 From Deep Learning to Intelligent Systems

Students have already developed systems such as

$$\text{Patient Features} \longrightarrow \text{Neural Network} \longrightarrow \text{Diabetes Prediction}.$$

For image data:

$$\text{Image} \longrightarrow \text{CNN} \longrightarrow \text{Class}.$$

For sequential data:

$$\text{Time Series} \longrightarrow \text{LSTM} \longrightarrow \text{Future Prediction}.$$

These are important intelligent-system components.
However, they primarily solve prediction problems.

2.1 Prediction Versus Knowledge

Consider the following questions:

What is diabetes?

Why does a CNN work well for images?

What is the difference between CNN and RNN?

What is an LSTM?

What is RAG?

Why does RAG need retrieval?

What is a Knowledge Graph?

A classifier or regressor does not directly answer these questions.
The problem has changed.
Instead of

$$x \to y,$$

we now have

$$\text{Question} \to \text{Knowledge} \to \text{Reasoning} \to \text{Answer}.$$

2.2 The Intelligent-System Perspective

A modern intelligent application can be represented as

$$\boxed{\text{Data} \to \text{Representation} \to \text{Knowledge} \to \text{Retrieval} \to \text{Model} \to \text{Decision or Conversation}}$$

The machine-learning model is therefore only one component.
The system may contain:

- data;

- databases;
- Knowledge Graphs;
- embeddings;
- retrieval algorithms;
- language models;
- user interfaces;
- evaluation mechanisms.

This broader view is the purpose of the course *Intelligent System Development*.

3 Knowledge Representation

Knowledge can be represented in several different forms.

3.1 Structured Knowledge

A conventional relational database stores information in tables.

A Knowledge Graph stores entities and their relationships.
For example:

$$(\text{CNN}, \text{is-a}, \text{Deep Learning Model})$$

$$(\text{CNN}, \text{used-for}, \text{Image Classification})$$

$$(\text{MNIST}, \text{used-by}, \text{CNN})$$

A Knowledge Graph therefore explicitly represents relationships.

3.2 Unstructured Knowledge

Much useful knowledge is stored in documents:

- lecture notes;
- PDF files;
- textbooks;
- laboratory instructions;
- web pages;
- technical reports;
- source code documentation.

For example:

> Convolutional neural networks are widely used for image classification because convolutional layers can learn local spatial patterns.

This sentence contains knowledge.
However, the relationship between

$$\text{CNN}$$

and

$$\text{Image Classification}$$

is not explicitly represented as a graph edge.

3.3 Why We Need Both

A Knowledge Graph is useful for explicit relationships:

$$\text{CNN} \rightarrow \text{used-for} \rightarrow \text{Image Classification.}$$

Documents are useful for detailed explanations.
Therefore a practical intelligent system may combine:

$$\boxed{\text{Knowledge Graph} + \text{Documents}}$$

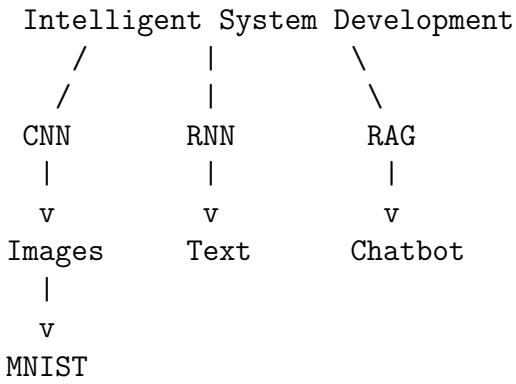
This combination becomes particularly useful when building domain-specific assistants.

4 Knowledge Graph Fundamentals

A Knowledge Graph contains:

- entities;
- relations;
- properties;
- connections.

A simple conceptual graph is:



4.1 Knowledge Graph as Triples

A common representation is

$$(\text{subject}, \text{relation}, \text{object}).$$

For example:

$$(\text{CNN}, \text{is-a}, \text{Deep Learning Model})$$

$$(\text{CNN}, \text{used-for}, \text{Image Classification})$$

$$(\text{MNIST}, \text{used-by}, \text{CNN}).$$

4.2 Python Implementation

We can use NetworkX for the first educational implementation.

```
import networkx as nx

G = nx.DiGraph()

G.add_node("CNN", type="model")
G.add_node("RAG", type="method")
G.add_node("MNIST", type="dataset")

```

```
G.add_edge(
    "CNN",
    "Image Classification",
    relation="used-for"
)

G.add_edge(
    "MNIST",
    "CNN",
    relation="used-by"
)

G.add_edge(
    "RAG",
    "Chatbot",
    relation="supports"
)

for source, target, data in G.edges(data=True):

    print(
        source,
        "--",
        data["relation"],
        "-->",
        target
    )
```

This simple example already provides explicit machine-readable knowledge.

5 Building a Course Knowledge Graph

We now construct a small Knowledge Graph for the course.

```
import networkx as nx

G = nx.DiGraph()

triples = [

    ("CNN",
     "is-a",
     "Deep Learning Model"),

    ("RNN",
     "is-a",
     "Deep Learning Model"),

    ("LSTM",
     "is-a",
     "Recurrent Model"),

    ("CNN",
     "used-for",
     "Image Classification"),

    ("LSTM",
     "used-for",
     "Time Series"),

    ("MNIST",
     "used-by",
     "CNN"),

    ("Stock Prediction",
     "uses",
     "LSTM"),

    ("RAG",
     "uses",
     "Vector Retrieval"),

    ("RAG",
     "uses",
     "LLM"),

    ("RAG",
     "supports",
     "Chatbot"),

    ("Knowledge Graph",
     "represents",
```

```
     "Entities"),

    ("Knowledge Graph",
     "represents",
     "Relations")
]

for subject, relation, object_ in triples:

    G.add_edge(
        subject,
        object_,
        relation=relation
    )

print(
    "Nodes:",
    G.number_of_nodes()
)

print(
    "Edges:",
    G.number_of_edges()
)
```

5.1 Graph Query

We can write a simple graph retrieval function.

```
def graph_facts(G, entity):

    facts = []

    if entity in G:

        for _, target, data in G.out_edges(
                entity,
                data=True):

            facts.append(
                f"{entity} "
                f"{data['relation']} "
                f"{target}."
            )

        for source, _, data in G.in_edges(
                entity,
                data=True):

            facts.append(
                f"{source} "
                f"{data['relation']} "
```

```
                f"{entity}."
            )

    return facts
```

Test:

```
facts = graph_facts(G, "RAG")

for fact in facts:
    print(fact)
```

Expected results include:

```
RAG uses Vector Retrieval.
RAG uses LLM.
RAG supports Chatbot.
```

The important principle is:

Graph retrieval follows explicit relationships.

6 Embeddings and Semantic Representation

Keyword search mainly compares words.

Semantic retrieval compares representations of meaning.

An embedding model is a function

$$f : \mathcal{T} \to \mathbb{R}^d,$$

where $\mathcal{T}$ is the space of text and d is the embedding dimension.

For a sentence,

$$f(\text{CNN classifies images}) = [x_1, x_2, \ldots, x_d].$$

The resulting vector is called an embedding.

6.1 Cosine Similarity

For two vectors x and y,

$$\text{sim}(x, y) = \frac{x^T y}{\|x\|_2 \|y\|_2}.$$

If the vectors are normalized,

$$\|x\|_2 = \|y\|_2 = 1,$$

then

$$\text{sim}(x, y) = x^T y.$$

The retrieval problem becomes:

$$q = f(\text{query}),$$

$$d_i = f(\text{document}_i),$$

and

$$i^\star = \arg\max_i q^T d_i.$$

6.2 First Experiment: TF-IDF

Before introducing neural embeddings, students should understand the retrieval idea using TF-IDF.

```
from sklearn.feature_extraction.text import (
    TfidfVectorizer
)

from sklearn.metrics.pairwise import (
    cosine_similarity
)

```

```
documents = [

    "CNN is used for image classification.",

    "LSTM is useful for sequential data.",

    "RAG retrieves knowledge before generation.",

    "House price prediction is a regression task."
]

query = (
    "How can a model classify images?"
)

vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(
    documents
)

q = vectorizer.transform(
    [query]
)

scores = cosine_similarity(
    q,
    X
)[0]

for i in scores.argsort()[::-1]:

    print(
        scores[i],
        documents[i]
    )
```

This establishes the central idea of retrieval before students move to neural embeddings.

7 Semantic Embeddings with Sentence Transformers

Now use a neural sentence embedding model.

```
from sentence_transformers import (
    SentenceTransformer
)

import numpy as np

model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)
```

Create the document embeddings:

```
texts = [

    "CNN is used for image classification.",

    "LSTM is useful for sequential data.",

    "RAG retrieves knowledge before generation.",

    "House price prediction is a regression task."
]

embeddings = model.encode(
    texts,
    normalize_embeddings=True
)

print(embeddings.shape)
```

Now embed a query:

```
query = (
    "How does image classification work?"
)

q = model.encode(
    [query],
    normalize_embeddings=True
)[0]

scores = embeddings @ q

top_k = 2

indices = np.argsort(
    scores
)[::-1][:top_k]

for i in indices:
```

```
19
20      print(
21          scores[i],
22          texts[i]
23      )
```

This is semantic retrieval.
The mathematical operation is

$$\boxed{\text{query} \rightarrow \text{embedding} \rightarrow \text{similarity} \rightarrow \text{top-}k}$$

8 Document Chunking

Suppose a lecture PDF contains 50 pages.

It is usually inefficient to represent the entire PDF as one retrieval unit.

Instead,

$$D \to \{C_1, C_2, \ldots, C_N\}.$$

Each C_i is a chunk.

8.1 Simple Chunking Algorithm

```python
def chunk_text(
    text,
    chunk_size=500,
    overlap=100
):

    words = text.split()

    chunks = []

    start = 0

    while start < len(words):

        end = start + chunk_size

        chunk = " ".join(
            words[start:end]
        )

        chunks.append(chunk)

        start += (
            chunk_size - overlap
        )

    return chunks
```

Test:

```
text = """
Deep learning uses neural networks
with multiple layers.

CNNs are particularly effective
for image data.

RNNs process sequential information.

LSTMs handle long-term dependencies.
```

```
11
12 RAG combines retrieval with
13 language generation.
14 """
15
16 chunks = chunk_text(
17     text,
18     chunk_size=20,
19     overlap=5
20 )
21
22 for i, chunk in enumerate(chunks):
23
24     print(
25         "\nCHUNK",
26         i
27     )
28
29     print(chunk)
```

8.2 Why Overlap?

Suppose an important sentence crosses a chunk boundary.

Without overlap, the meaning may be divided.

Therefore:

$$\text{chunk} + \text{overlap}$$

helps preserve local context.

Typical experiments should compare different values.

For example:

$$100, \quad 300, \quad 500, \quad 800.$$

The optimal value depends on the document and application.

9 Document Metadata

A good RAG system should not store only text.

It should also store metadata.

For example:

```
chunk = {

    "text":
        "CNNs are useful for image classification.",

    "source":
        "lecture01.pdf",

    "page":
        7,

    "topic":
        "CNN",

    "course":
        "Intelligent System Development"
}
```

Metadata supports:

- source citation;
- filtering;
- debugging;
- evaluation;
- access control;
- document management.

For example, the system may retrieve only documents from:

$$\text{course} = \text{ISD}$$

and

$$\text{topic} = \text{RAG}.$$

This is much more powerful than treating all text as an undifferentiated collection.

9.1 A Better Internal Representation

A practical chunk object can be:

```
chunk = {

    "id": "lecture08_page07_chunk03",

    "text": "...",

    "source": "lecture08.pdf",

    "page": 7,

    "topic": "RAG",

    "embedding": None
}
```

The embedding is generated during ingestion.
At query time, we should not recompute embeddings for every stored document.
This leads to the system principle:

Index once; retrieve many times.

10 What is Retrieval-Augmented Generation?

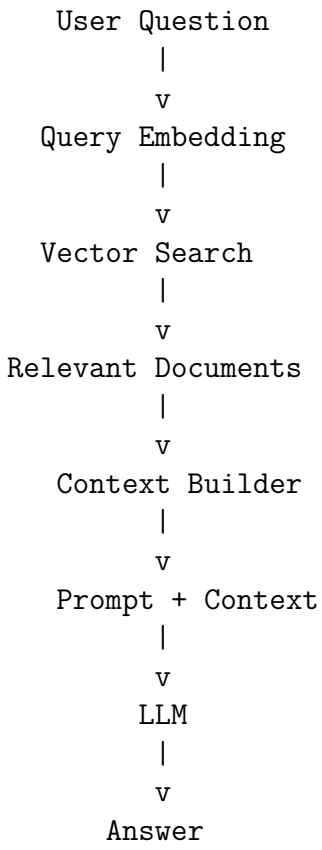
RAG means

$$\boxed{\text{Retrieval-Augmented Generation}}$$

and combines:

$$\text{Retrieval} + \text{Context} + \text{Generation}.$$

The basic architecture is:



Without RAG:

$$\text{Question} \rightarrow \text{LLM} \rightarrow \text{Answer}.$$

With RAG:

$$\text{Question} \rightarrow \text{Retrieve Evidence} \rightarrow \text{LLM} + \text{Evidence} \rightarrow \text{Answer}.$$

10.1 Why RAG?

RAG is useful when knowledge is:

- domain-specific;
- private;
- frequently updated;

- too large to put into every prompt;
- stored in documents.

For example, a university course may have:

100+

lecture files and assignments.
Instead of retraining the LLM, we can update the retrieval index.

11 Building the First RAG Retriever

We now implement a complete retrieval module.

```
import numpy as np

from sentence_transformers import (
    SentenceTransformer
)

model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)

documents = [

    {
        "text":
        "CNNs are deep learning models "
        "commonly used for image classification.",

        "source":
        "lecture01"
    },

    {
        "text":
        "RNNs process sequential information "
        "and are useful for sequence modelling.",

        "source":
        "lecture02"
    },

    {
        "text":
        "LSTMs are recurrent models designed "
        "to handle long-term dependencies.",

        "source":
        "lecture02"
    },

    {
        "text":
        "RAG retrieves relevant information "
        "before an LLM generates an answer.",

        "source":
        "lecture03"
    }
]
```

```
49
50 texts = [
51     d["text"]
52     for d in documents
53 ]
54
55 embeddings = model.encode(
56     texts,
57     normalize_embeddings=True
58 )
```

11.1 Retrieval Function

```
1  def retrieve(
2      query,
3      top_k=3
4  ):
5
6      q = model.encode(
7          [query],
8          normalize_embeddings=True
9      )[0]
10
11     scores = embeddings @ q
12
13     indices = np.argsort(
14         scores
15     )[::-1][:top_k]
16
17     results = []
18
19     for i in indices:
20
21         results.append({
22
23             "text":
24                 documents[i]["text"],
25
26             "source":
27                 documents[i]["source"],
28
29             "score":
30                 float(scores[i])
31         })
32
33     return results
```

Test:

```
1 results = retrieve(
2     "How does RAG use retrieved information?",
```

```
    top_k=3
)

for result in results:

    print(
        result["score"],
        result["source"]
    )

    print(
        result["text"]
    )
```

The retriever is now an independent component.

12 Constructing the RAG Context

Retrieval produces several relevant chunks.

The chunks must be converted into a context.

```
def build_context(results):

    blocks = []

    for result in results:

        blocks.append(

            f"[Source: "
            f"{result['source']}]\n"
            f"{result['text']}"

        )

    return "\n\n".join(
        blocks
    )
```

Now create the prompt.

```
def build_prompt(
    question,
    context
):

    return f"""
You are a course assistant.

Answer the question using only
the supplied context.

If the context does not contain
enough information, say that the
knowledge base does not provide
sufficient information.

CONTEXT:

{context}

QUESTION:

{question}

ANSWER:
"""
```

The complete retrieval preparation function is:

```python
def prepare_rag(question):

    results = retrieve(
        question,
        top_k=3
    )

    context = build_context(
        results
    )

    prompt = build_prompt(
        question,
        context
    )

    return prompt, results
```

The architecture is now:

$$\boxed{\text{Question} \rightarrow \text{Retriever} \rightarrow \text{Context} \rightarrow \text{Prompt}}$$

The final generation component can be connected afterward.

13 Connecting an LLM

The LLM is responsible for generating natural-language answers from the retrieved evidence.

Conceptually:

```
question = "What is RAG?"

prompt, results = prepare_rag(
    question
)

response = llm.generate(
    prompt
)

print(response)
```

The actual API depends on the provider.

To keep the architecture provider-independent, students can define:

```
def generate_answer(prompt):

    # Connect this function to
    # an approved LLM API or
    # a local language model.

    return (
        "LLM-generated answer"
    )
```

Then:

```
def rag_answer(question):

    prompt, results = (
        prepare_rag(question)
    )

    answer = generate_answer(
        prompt
    )

    return {

        "answer":
            answer,

        "sources":
            results
    }
```

The returned object contains both:

Answer

and

Evidence.

This is an important design principle.
A chatbot should not merely say:

> RAG retrieves information before generation.

It should preferably also say where the information came from.

14 Complete Command-Line RAG Prototype

The following program combines the main components.

```
import numpy as np

from sentence_transformers import (
    SentenceTransformer
)

model = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)

documents = [

    {
        "text":
        "CNNs are deep learning models "
        "commonly used for image classification.",

        "source":
        "lecture01"
    },

    {
        "text":
        "RNNs process sequential information.",

        "source":
        "lecture02"
    },

    {
        "text":
        "LSTMs handle long-term dependencies "
        "in sequential data.",

        "source":
        "lecture02"
    },

    {
        "text":
        "RAG retrieves relevant information "
        "before an LLM generates an answer.",

        "source":
        "lecture03"
    }
]

```

```
texts = [
    d["text"]
    for d in documents
]

embeddings = model.encode(
    texts,
    normalize_embeddings=True
)


def retrieve(
    query,
    top_k=3
):

    q = model.encode(
        [query],
        normalize_embeddings=True
    )[0]

    scores = embeddings @ q

    indices = np.argsort(
        scores
    )[::-1][:top_k]

    return [

        {
            "text":
                documents[i]["text"],

            "source":
                documents[i]["source"],

            "score":
                float(scores[i])
        }

        for i in indices
    ]


def build_prompt(
    question,
    results
):

    context = "\n\n".join(

        f"[{r['source']}] "
```

```
        f"{r['text']}"

        for r in results
    )

    return f"""
You are a course assistant.

Use only the supplied context.

If the context is insufficient,
say so explicitly.

CONTEXT:

{context}

QUESTION:

{question}

ANSWER:
"""


def generate_answer(prompt):

    # Replace this function with
    # approved LLM inference.

    return (
        "Generate the grounded "
        "answer here."
    )


while True:

    question = input(
        "\nStudent: "
    )

    if question.lower() in {
        "exit",
        "quit"
    }:

        break

    results = retrieve(
        question
    )
```

```
153
154     prompt = build_prompt(
155         question,
156         results
157     )
158
159     answer = generate_answer(
160         prompt
161     )
162
163     print(
164         "\nAssistant:",
165         answer
166     )
167
168     print(
169         "\nSources:"
170     )
171
172     for result in results:
173
174         print(
175             f"- {result['source']} "
176             f"(score="
177             f"{result['score']:.3f})"
178         )
```

This is already a complete RAG architecture even though the generation function is deliberately abstracted.

15 Knowledge Graph + RAG

Vector retrieval and graph retrieval solve different problems.

Vector retrieval asks:

> Which passages are semantically similar to the question?

Graph retrieval asks:

> Which entities and relationships are explicitly connected?

Therefore a hybrid system can be represented as

$$\text{Question} \rightarrow \begin{cases} \text{Graph Retrieval} \\ \text{Vector Retrieval} \end{cases} \rightarrow \text{Context Fusion} \rightarrow \text{LLM}.$$

15.1 Graph Retrieval

```python
def graph_retrieve(
    G,
    entity
):

    facts = []

    if entity in G:

        for _, target, data in (
                G.out_edges(
                    entity,
                    data=True
                )):

            facts.append(

                f"{entity} "
                f"{data['relation']} "
                f"{target}."

            )

        for source, _, data in (
                G.in_edges(
                    entity,
                    data=True
                )):

            facts.append(

                f"{source} "
                f"{data['relation']} "
```

```
                f"{entity}."

            )

    return facts
```

15.2 Combining the Two Sources

Suppose the question is:

> What is RAG and what does it use?

Graph retrieval may produce:

```
RAG uses Vector Retrieval.
RAG uses LLM.
RAG supports Chatbot.
```

Vector retrieval may produce a paragraph explaining how RAG works.
Combine them:

```
question = (
    "What is RAG and what does it use?"
)

graph_context = "\n".join(

    graph_retrieve(
        G,
        "RAG"
    )
)

vector_results = retrieve(
    question,
    top_k=2
)

vector_context = build_context(
    vector_results
)

combined_context = (

    "GRAPH FACTS:\n"
    + graph_context
    + "\n\n"
    + "DOCUMENT EVIDENCE:\n"
    + vector_context
)

print(combined_context)
```

The LLM receives richer evidence.

16 Concrete Application: Personal Course Assistant

The recommended final application for students is a **Personal Course Assistant**.

The student creates a chatbot using his or her own course materials.

Possible knowledge sources include:

- lecture PDFs;
- assignment specifications;
- laboratory instructions;
- course notes;
- programming examples;
- project documentation.

The system architecture is:

$$\text{Course Documents} \rightarrow \text{Text Extraction} \rightarrow \text{Chunking} \rightarrow \text{Embedding} \rightarrow \text{Index}.$$

At query time:

$$\text{Question} \rightarrow \text{Embedding} \rightarrow \text{Search} \rightarrow \text{Context} \rightarrow \text{LLM} \rightarrow \text{Answer}.$$

16.1 Suggested Project Directory

```
course_rag/
|
+-- app.py
|
+-- ingest.py
|
+-- retriever.py
|
+-- graph.py
|
+-- data/
|   |
|   +-- lectures/
|   +-- assignments/
|   +-- notes/
|
+-- index/
|
+-- evaluation/
|   |
|   +-- questions.csv
|
+-- requirements.txt
```

This organization teaches students that an intelligent system is a software system, not just a notebook.

17 Building a Chat Interface

A simple Streamlit interface can be implemented as follows.

```
import streamlit as st

st.set_page_config(
    page_title="Course RAG Assistant"
)

st.title(
    "Intelligent System Development "
    "Assistant"
)

question = st.chat_input(
    "Ask a question about the course..."
)

if question:

    st.chat_message(
        "user"
    ).write(question)

    results = retrieve(
        question,
        top_k=3
    )

    context = build_context(
        results
    )

    prompt = build_prompt(
        question,
        context
    )

    answer = generate_answer(
        prompt
    )

    st.chat_message(
        "assistant"
    ).write(answer)

    st.subheader(
        "Retrieved Sources"
    )

    for result in results:
```

```
49
50          st.write(
51
52              f"{result['source']} "
53              f"(score="
54              f"{result['score']:.3f})"
55
56          )
57
58          st.write(
59              result["text"]
60          )
```

Run the application:

```
1 streamlit run app.py
```

The interface should show:

1. the student's question;
2. the generated answer;
3. the retrieved evidence;
4. source information.

This makes the retrieval process visible and therefore useful for education and debugging.

18 Hallucination and Grounding

Large language models can generate fluent but unsupported information.

Without retrieval:

$$\text{Question} \rightarrow \text{LLM} \rightarrow \text{Plausible Answer}.$$

With RAG:

$$\text{Question} \rightarrow \text{Evidence} \rightarrow \text{LLM} \rightarrow \text{Grounded Answer}.$$

However:

> RAG reduces hallucination but does not guarantee zero hallucination.

18.1 Three Important Failure Modes

18.1.1 Retrieval Failure

The correct information exists, but the retriever does not find it.

$$\text{Knowledge exists} \not\Rightarrow \text{Knowledge retrieved}.$$

Possible causes:

- bad chunking;
- poor embedding model;
- ambiguous query;
- insufficient top-k;
- missing metadata.

18.1.2 Generation Failure

The correct evidence is retrieved, but the LLM misunderstands it.

18.1.3 Knowledge Gap

The answer does not exist in the knowledge base.

The system should then explicitly say:

> The available knowledge base does not contain sufficient information to answer this question.

18.2 Grounding Instruction

A useful system instruction is:

```
Answer only from the supplied context.

If the context is insufficient,
say that the knowledge base does not
contain enough information.

Do not invent facts.
```

This is a system-design requirement rather than merely a prompt engineering trick.

19 Evaluation of the RAG System

Students should not evaluate their chatbot by saying:

> "It works because it gives an answer."

An intelligent system must be evaluated systematically.

19.1 Test Set

Create at least 20 questions.

Example:

ID	Question	Expected Evidence
Q1	What is CNN?	CNN lecture
Q2	What is RAG?	RAG lecture
Q3	Why is chunking needed?	RAG lecture
Q4	What is LSTM?	LSTM lecture
Q5	Compare CNN and RNN.	CNN/RNN lectures
Q6	Why does RAG use retrieval?	RAG lecture
Q7	What is a Knowledge Graph?	KG lecture
Q8	How are triples represented?	KG lecture

The final project should use at least 20 questions.

19.2 Retrieval Recall

A simple retrieval metric is

$$\mathit{Recall@}k = \frac{\text{questions with relevant evidence in top } k}{\text{total questions}}.$$

For example, if 18 of 20 questions retrieve relevant evidence in the top three results:

$$\mathit{Recall@}3 = \frac{18}{20} = 0.90.$$

19.3 Answer Evaluation

Use a simple 0–2 scale:

$$0 = \text{incorrect},$$

$$1 = \text{partially correct},$$

$$2 = \text{correct}.$$

Evaluate:

- correctness;

- relevance;
- completeness;
- groundedness;
- source correctness.

20 Final Student Project

The recommended final project is:

> **Develop a Domain-Specific Intelligent Chat System using Knowledge Graph and RAG.**

Students can select a domain such as:

- university assistant;
- course assistant;
- e-commerce;
- programming assistant;
- travel;
- books;
- movies;
- sports;
- finance;
- stock-market education;
- personal study assistant.

20.1 Minimum Architecture

The minimum architecture is:

$$\text{Knowledge Sources} \rightarrow \text{Chunking} \rightarrow \text{Embedding} \rightarrow \text{Retriever} \rightarrow \text{LLM} \rightarrow \text{Chat}.$$

20.2 Advanced Architecture

For stronger students:

$$\text{Question} \rightarrow \begin{cases} \text{Knowledge Graph} \\ \text{Vector Database} \end{cases} \rightarrow \text{Evidence Fusion} \rightarrow \text{LLM} \rightarrow \text{Answer + Sources}.$$

20.3 Required Deliverables

Each project should contain:

1. problem definition;
2. domain description;
3. knowledge sources;
4. Knowledge Graph;
5. RAG architecture;
6. source code;
7. user interface;
8. test questions;
9. quantitative evaluation;
10. demonstration;
11. final report.

21 Recommended Student Report

Students should use the following report structure.

21.1 1. Introduction

Explain:

- the application problem;
- why a chatbot is useful;
- why conventional keyword search is insufficient;
- why RAG is appropriate.

21.2 2. Domain and Knowledge Sources

Describe:

- the selected domain;
- data sources;
- document types;
- data volume.

21.3 3. Knowledge Graph

Report:

- entities;
- relations;
- example triples;
- graph visualization;
- graph retrieval.

21.4 4. RAG Architecture

Describe:

$$\text{Documents} \rightarrow \text{Chunks} \rightarrow \text{Embeddings} \rightarrow \text{Retrieval} \rightarrow \text{Generation}.$$

21.5 5. Implementation

Explain:

- ingestion;
- chunking;
- embeddings;
- retrieval;
- graph queries;
- prompt construction;
- LLM;
- user interface.

21.6 6. Experiments

Students should vary at least one parameter:

$$k, \quad \text{chunk size}, \quad \text{embedding model}, \quad \text{KG on/off}.$$

21.7 7. Evaluation

Report retrieval and answer quality.

21.8 8. Deployment

Demonstrate the working system.

21.9 9. Limitations

Discuss:

- incomplete knowledge;
- retrieval errors;
- hallucination;
- stale information;
- computational cost;
- privacy.

21.10 10. Conclusion

Explain what the system achieved and what should be improved.

22 Laboratory Exercises

Exercise 1: Build a Knowledge Graph

Create at least:

$$20 \text{ entities}$$

and

$$30 \text{ relationships.}$$

Store the triples in CSV format:

```
subject,relation,object
CNN,is-a,Deep Learning Model
CNN,used-for,Image Classification
MNIST,used-by,CNN
```

Students should implement:

- graph construction;
- graph visualization;
- one-hop query;
- two-hop query.

Exercise 2: Document Retrieval

Collect at least five documents.

Perform:

1. text extraction;
2. chunking;
3. embedding;
4. semantic retrieval.

Compare:

$$k = 1, \quad k = 3, \quad k = 5.$$

Exercise 3: Build RAG

Connect retrieved chunks to an LLM.

Require:

- grounded answer;
- source citation;
- “not enough information” behavior.

Exercise 4: KG + RAG

For questions involving known entities:

1. identify entities;
2. retrieve graph facts;
3. retrieve document evidence;
4. combine evidence;
5. generate the answer.

Exercise 5: Evaluation

Compare:

LLM only

versus

Vector RAG

versus

Knowledge Graph + RAG.

Use at least 20 questions.

Exercise 6: Deployment

Create a Streamlit application.

Students must demonstrate:

- question;
- answer;
- retrieved evidence;
- source;
- at least one graph-based answer.

23 Debugging and System Design Checklist

23.1 Problem 1: Poor Retrieval

Possible causes:

- chunks are too large;
- chunks are too small;
- embedding model is unsuitable;
- question is ambiguous;
- top-k is too small.

Students should perform controlled experiments rather than changing everything simultaneously.

23.2 Problem 2: Good Retrieval, Bad Answer

Inspect the prompt.

The LLM should receive:

$$\text{Question} + \text{Relevant Evidence} + \text{Clear Instruction}.$$

The model should not be asked to infer facts that are absent from the evidence.

23.3 Problem 3: Hallucination

Test questions for which the answer is intentionally absent.

Expected behavior:

> The knowledge base does not contain sufficient information.

This is an important evaluation experiment.

23.4 Problem 4: Graph Retrieval Too Broad

Do not return the entire Knowledge Graph.

Start from the relevant entity and retrieve:

$$\text{1-hop}$$

or

$$\text{2-hop}$$

relationships.

23.5 Problem 5: Slow Retrieval

Document embeddings should be computed during ingestion.

Do not recompute all document embeddings for every question.

$$\boxed{\text{Ingestion time} \neq \text{Query time}}$$

23.6 Problem 6: Security

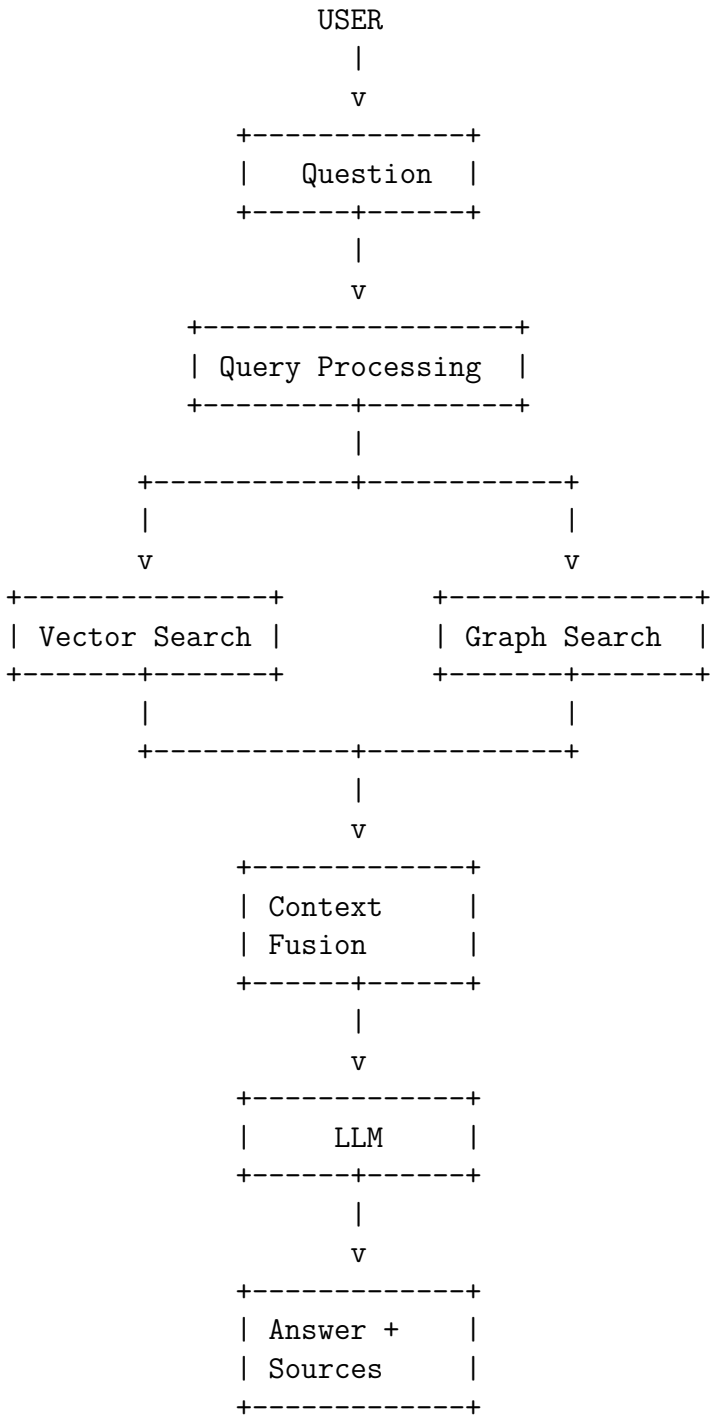
Students must not:

- hard-code API keys;
- upload private documents without permission;
- expose confidential information;
- assume that every retrieved document is trustworthy.

Use environment variables for API credentials.

24 Final System Architecture

Students should finish the lecture able to draw the following architecture.



The architecture contains several independent components:

Knowledge
Representation
Retrieval
Generation
Interface
Evaluation

This decomposition is important for Intelligent System Development.

25 From ML and DL to RAG-Based Intelligent Systems

The students' learning progression can now be summarized as

ML → DL → Prediction → Representation → Knowledge → Retrieval → RAG → Conversation → Inte

Earlier in the course, students asked:

> Can a neural network predict the target?

Now they ask:

> Can I build a system that understands a user's question, finds appropriate knowledge, and produces a grounded answer?

This is a substantially broader engineering problem.

25.1 The Main Concept

The most important message of the lecture is:

> **The LLM is one component inside an intelligent information system.**

A strong intelligent system requires:

$$\text{Data} + \text{Knowledge} + \text{Retrieval} + \text{Model} + \text{Interface} + \text{Evaluation}.$$

25.2 Student Final Checklist

- ☐ Domain clearly defined.
- ☐ Knowledge sources documented.
- ☐ Knowledge Graph constructed.
- ☐ At least 20 entities identified.
- ☐ At least 30 relations constructed.
- ☐ Documents divided into chunks.
- ☐ Embeddings generated.
- ☐ Vector retrieval implemented.
- ☐ RAG prompt implemented.
- ☐ LLM connected.
- ☐ Sources displayed.
- ☐ At least 20 evaluation questions created.

- ☐ Retrieval quality measured.
- ☐ Answer quality measured.
- ☐ KG + RAG compared with vector RAG.
- ☐ Chat interface deployed.
- ☐ Limitations discussed.

Conclusion

The purpose of this lecture is not simply to teach students a new keyword called "RAG."

The purpose is to move students from isolated machine-learning experiments toward complete intelligent-system development.

The progression is:

Learn a Model → Build a Component → Integrate Components → Build a System → Evaluate the Sy

Knowledge Graphs provide explicit structured relationships.
Embeddings provide semantic representations.
Vector retrieval provides access to relevant documents.
RAG connects retrieval with language generation.
A conversational interface makes the system accessible to users.
Therefore:

Knowledge Graph + Vector Retrieval + LLM = Knowledge-Based Intelligent Chat System

The final student project should demonstrate that the student can move beyond training a neural network and can instead design, implement, evaluate, and deploy an integrated intelligent system.